



Práctica 8

DISPLAYS MULTIPLEXADOS

Sistemas en Chips

- ▶ **Prof. José Juan Pérez Pérez**
- ▶ **Grupo: 6CVI**
- ▶ **Equipo I:**
 - ▶ Ayala Fuentes Sunem Gizeht
 - ▶ Guevara Badillo Areli Alejandra
 - ▶ León Pérez May Carolina

Introducción

Interrupciones Externas en Microcontroladores AVR

Las interrupciones son eventos de hardware que suspenden temporalmente la ejecución del programa principal para que el procesador atienda una tarea prioritaria (Subrutina de Servicio de Interrupción o ISR). Una vez completada la tarea, el procesador retoma la ejecución exactamente donde se quedó.

En la familia ATmega, existen interrupciones generadas internamente (por periféricos) y externamente (por cambios de voltaje en pines específicos).

La Interrupción INT2

Mientras que INT0 e INT1 son las interrupciones externas estándar configurables por nivel o por flanco (subida/bajada), la interrupción **INT2** posee características particulares en muchos modelos AVR:

- **Configuración Simplificada:** Generalmente, INT2 se configura para dispararse únicamente por flanco (edge-triggered), ya sea de subida o de bajada.
- **Vector Independiente:** Posee su propia dirección en la tabla de vectores de interrupción, lo que permite ejecutar un código totalmente distinto al de INT0 o INT1 sin necesidad de verificar qué pin fue activado.
- **Aplicación:** Es ideal para funciones de control de estado crítico, como "Inicio/Parada" o eventos de emergencia, ya que su activación es inmediata y prioritaria sobre el bucle principal (main).

Enmascaramiento de Interrupciones (Control de Pausa)

Para detener una animación controlada por hardware (Timer) sin apagar el microcontrolador, se utiliza la técnica de **enmascaramiento**.

Esto consiste en modificar los registros de control de interrupciones (TIMSK o equivalentes) para habilitar o deshabilitar selectivamente la interrupción del Timer de animación. Al "enmascarar" (deshabilitar) esta interrupción, el contador sigue corriendo, pero la CPU ignora el evento de desbordamiento, congelando efectivamente la actualización de la posición del texto (Pausa).

Temporizadores (Timers/Counters)

Un Timer es un periférico interno que cuenta ciclos de reloj de forma independiente a la Unidad Central de Procesamiento (CPU). Esto permite medir el tiempo o generar eventos periódicos sin que la CPU tenga que estar "ocupada" ejecutando bucles de retardo (*busy-waiting*).

Modos de Operación

Para aplicaciones de visualización y animación, el modo más común es el Modo Normal con Interrupción por Desbordamiento (Overflow):

- El contador inicia en 0 y cuenta hacia arriba hasta su valor máximo (255 para 8 bits, 65535 para 16 bits).
- Al pasar del máximo a 0 nuevamente, se genera una bandera de "Desbordamiento" (Overflow Flag).
- Si la interrupción está habilitada, el código salta a la rutina correspondiente.

Prescalador (Prescaler)

Es un divisor de frecuencia que permite al Timer contar más lento que la velocidad del reloj del sistema (f_{osc}).

$$f_{timer} = \frac{f_{osc}}{N}$$

Donde N es el factor de división (1, 8, 64, 256, 1024). Esto es fundamental para generar tiempos largos (visibles al ojo humano) a partir de frecuencias de reloj muy altas (MHz).

Multitarea Aparente (Uso de dos Timers)

En sistemas de visualización avanzados, se suelen emplear dos timers con propósitos distintos:

1. **Timer de Alta Frecuencia:** Se encarga del Multiplexado. Debe ser lo suficientemente rápido (>60 Hz) para refrescar la matriz de LEDs sin parpadeo visible.
2. **Timer de Baja Frecuencia:** Se encarga de la Animación. Define la velocidad a la que se mueve o cambia el texto.

Visualización Dinámica: Desplazamiento de Mensajes (Scrolling)

El desplazamiento o *scroll* es una técnica visual utilizada cuando el mensaje a mostrar es más largo que la capacidad física del display (matriz de LEDs).

Concepto de Buffer de Video (RAM)

Dado que la memoria de programa (Flash) es lenta o no modificable en tiempo de ejecución para este fin, se utiliza una sección de la memoria SRAM como "pizarra" o Buffer de Video.

1. Los patrones de los caracteres se copian desde la memoria permanente a la SRAM.
2. La rutina de visualización lee únicamente los datos que están en la SRAM.

Ventana Deslizante y Punteros

Para crear el efecto de movimiento, no se mueven los LEDs físicamente, sino que se manipula la dirección de memoria que se está leyendo.

- **Puntero de Inicio:** Se define una variable que indica cuál es la primera columna de datos a mostrar en la matriz.
- **Animación:** Periódicamente (controlado por el Timer de animación), este puntero se incrementa.
- **Efecto:** Al mostrar las columnas n a $n+7$, y luego las columnas $n+1$ a $n+8$, el ojo humano percibe que el texto se desplaza hacia la izquierda.

Planteamiento del ejercicio

El objetivo de la práctica consiste en diseñar e implementar un sistema de visualización dinámica utilizando una **Matriz de LEDs de 8x8** controlada por el microcontrolador **ATmega8535**.

El sistema debe tener la capacidad de desplegar mensajes de texto con efecto de desplazamiento horizontal (*scrolling*) y permitir la interacción en tiempo real mediante interrupciones externas.

Las especificaciones funcionales son:

1. **Visualización:** Mostrar tres palabras diferentes almacenadas en la memoria del programa.
2. **Control de Contenido:** Utilizar las interrupciones externas **INT0** e **INT1** para alternar entre los mensajes almacenados.
3. **Control de Animación:** Utilizar la interrupción externa **INT2** para pausar y reanudar el desplazamiento del texto.

Desarrollo

Material

Para la realización de esta práctica se requieren los siguientes componentes y herramientas:

- Microcontrolador ATmega8535.
- 1 Matriz de LEDs de 8x8 (configuración Cátodo).
- 4 Pulsadores (*Push-buttons*).
- 1 Resistencias de 1kΩ.
- 3 capacitores de 1μF a 16V.
- Fuente de alimentación de 5V DC.
- 2 placas de pruebas.
- Cables de conexión.

Código

```
.include "m8535def.inc"
.def aux = r16
.def col = r17
.def cero = r19
.def point = r20

; Macro para cargar patrones desde Flash a SRAM
.macro txt
    ldi zh,high(@0<<1)
    ldi zl,low(@0<<1)
    clr yh
    ldi yl,$60
etq1:
    lpm aux,z+
    st y+,aux
    cpi yl,$68
    brne etq1
    ; (Se repite el bloque de carga para las siguientes letras...)
    ; [Código abreviado por claridad en el reporte, ver implementación completa]
.endm

reset:
    rjmp main
    rjmp texto1    ; Vector INT0
    rjmp texto2    ; Vector INT1
    .org $008
    rjmp corre     ; Vector Timer1 Overflow
    rjmp barre     ; Vector Timer0 Overflow
    .org $012
    rjmp stop      ; Vector INT2

main:
    ldi aux,low(ramend)
    out spl,aux
    ldi aux,high(ramend)
    out sph, aux
    clr cero
    rcall config_io
    rcall texto0    ; Carga texto inicial
```

```

; Configuración inicial de punteros y barrido
clr zh
ldi point,$60
mov zl,point
ldi col,1
com col
out portc,col
com col
ld aux,z
out porta,aux

uno:
nop
nop
rjmp uno      ; Bucle infinito (Idle)

config_io:
; Configuración de Puertos A, B, C, D y Timers
ser aux
out ddra,aux
out portb,aux
out ddrc,aux
out portd,aux
ldi aux,2
out tccr0,aux ; Prescaler Timer0
ldi aux,2
out tccr1b,aux ; Prescaler Timer1
ldi aux,$01
out tmsk,aux ; Habilita interrupción Timer0
ldi r18,193 ; Valor de carga para TCNT0
ldi aux,$0a ; Configuración de flancos INT0/INT1
out mcucr,aux
ldi aux,$e0 ; Habilita INT0, INT1, INT2
out gicr,aux
sei ; Habilita interrupciones globales
ret

; Rutinas de servicio de interrupción (ISR) y subrutinas
texto0: txt S,U,N,E,M : ret
texto1: txt A,R,E,L,I : clr zh : mov zl,point : reti
texto2: txt M,A,Ye,Dp,Dp : clr zh : mov zl,point : reti

barre: ; Multiplexación de la matriz (Timer0)
out tcnt0,r18
out porta,cero
inc zl
lsl col
brne dos
ldi col,1
mov zl,point
dos:
com col
out portc,col
com col
ld aux,z
out porta,aux
reti

corre: ; Animación de desplazamiento (Timer1)
inc point
cpi point,$68+$08+$08+$08+$08
brne ok
ldi point,$60
ok:
reti

```

```

stop:    ; Pausa/Reanudar (INT2)
        push aux
        push col
        in aux,timsk
        ldi col,$04    ; Máscara para TOIE1
        eor aux,col    ; Toggle del bit de habilitación
        out timsk,aux
        pop col
        pop aux
        reti

; Tablas de datos (Mapas de bits de caracteres)
S: .db $00,$4c,$92,$92,$92,$92,$92,$64
U: .db $00,$7e,$80,$80,$80,$80,$80,$7e
N: .db $00,$fe,$04,$08,$10,$20,$40,$fe
E: .db $00,$fe,$92,$92,$92,$92,$92,$82
M: .db $00,$fe,$04,$08,$10,$08,$04,$fe
A: .db $00,$f8,$24,$22,$22,$22,$24,$f8
R: .db $00,$fc,$12,$12,$32,$72,$d2,$8c
I: .db $00,$00,$82,$82,$fe,$82,$82,$00
L: .db $00,$fe,$80,$80,$80,$80,$80,$80
Ye: .db $00,$02,$04,$08,$f0,$08,$04,$02
Dp: .db $c0,$c0,$00,$c0,$c0,$00,$c0,$c0

```

Ilustración 1. Código implementado.

Solución

La solución implementada gestiona la complejidad del sistema dividiendo las tareas mediante interrupciones:

1. **Gestión de Memoria (Buffer SRAM):** Se definió un área de memoria en la SRAM (dirección \$60) que actúa como un buffer de video. Las interrupciones **INT0** e **INT1** invocan la macro txt, la cual transfiere los patrones de bits de las palabras seleccionadas desde la memoria Flash hacia este buffer.
2. **Multiplexación (Timer0):** El Timer0 genera interrupciones de alta frecuencia. En cada interrupción (barre), se apaga el puerto de datos, se rota el bit de selección de columna en el Puerto C y se envía el nuevo dato de fila desde la SRAM al Puerto A.
3. **Animación (Timer1):** El Timer1 genera interrupciones periódicas (corre) que incrementan el puntero de lectura (point). Esto desplaza la "ventana" de datos que lee la rutina de barrido, creando el efecto visual de movimiento.
4. **Control de Flujo (INT2):** La rutina stop manipula directamente el registro TIMSK. Al hacer una operación XOR con el bit correspondiente a la interrupción del Timer1, se habilita o deshabilita la rutina de animación, logrando pausar el texto sin detener el barrido (la matriz sigue encendida).

Circuito

La interfaz de hardware se establece de la siguiente manera:

- **Puerto A (Bus de Datos):** Conectado a las filas de la matriz de LEDs. Envía el patrón de bits correspondiente a la columna activa.
- **Puerto C (Bus de Control):** Conectado a las columnas de la matriz. Realiza el barrido secuencial.
- **Puerto D y B (Entradas de Control):**
 - **PD2 (INT0):** Pulsador para seleccionar el Mensaje 1.
 - **PD3 (INT1):** Pulsador para seleccionar el Mensaje 2.
 - **PB2 (INT2):** Pulsador para la función de Pausa/Inicio.

Diagrama del circuito

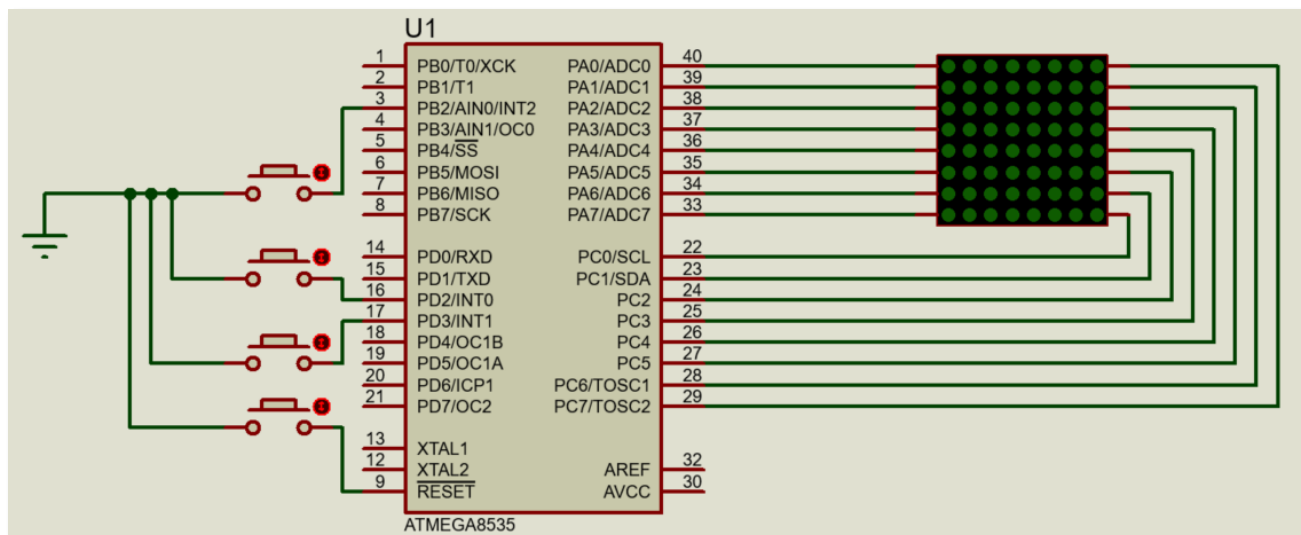


Ilustración 2. Esquema del circuito en proteus.

Simulación

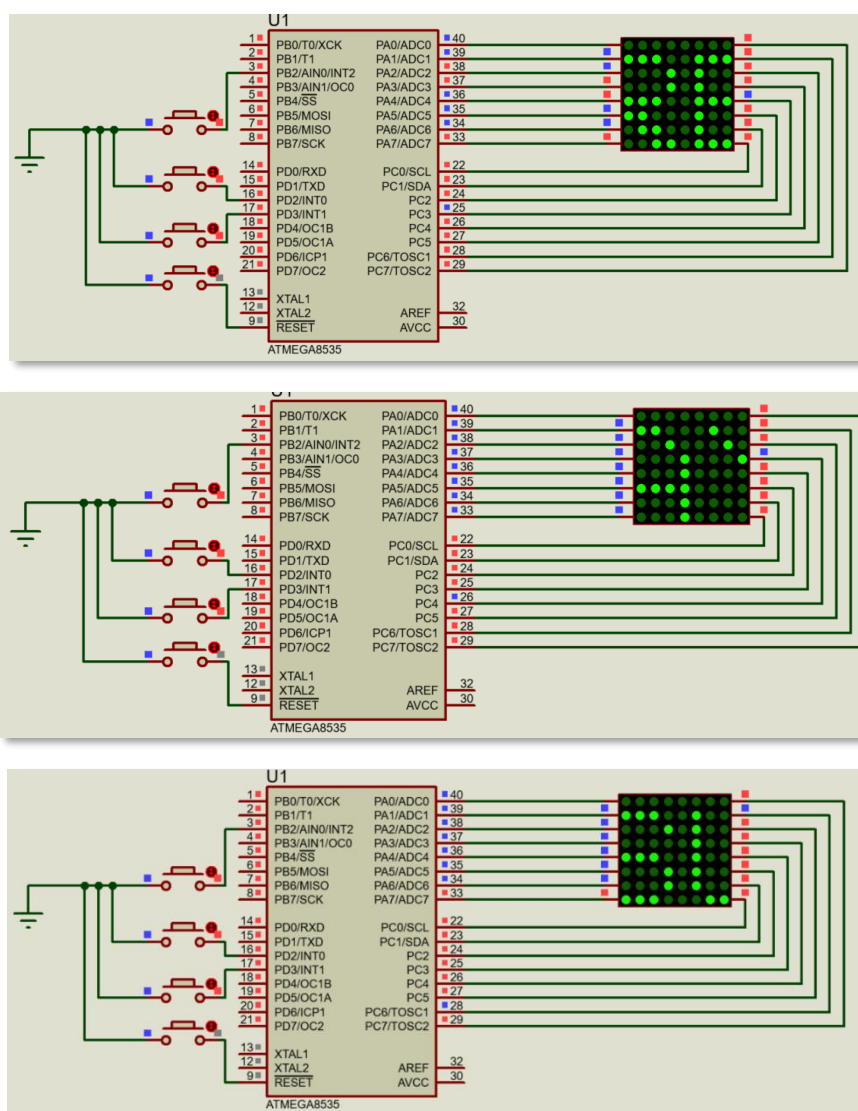


Ilustración 3. Simulación del funcionamiento en proteus.

Circuito Armado

Ilustración 4. Armado físico del circuito funcionando.

Conclusiones

Ayala Fuentes Sunem Gizeht

Guevara Badillo Areli Alejandra

León Pérez May Carolina

Referencias

- ▶ Circuit Digest. (s.f.). *Scrolling text on 8x8 LED matrix using AVR*. Recuperado de <https://circuitdigest.com/microcontroller-projects/scrolling-text-on-8x8-led-matrix-using-atmega32-avr>
- ▶ Raja, D. (2025, June 18). 8x8 LED Matrix Interfacing with AVR Microcontroller. Recuperado de <https://circuitdigest.com/microcontroller-projects/8x8-led-matrix-with-avr-atmega8>
- ▶ Mazidi, M. A., Naimi, S., & Naimi, S. (2011). *The AVR microcontroller and embedded systems: Using Assembly and C*. Upper Saddle River, NJ: Prentice Hall.
- ▶ Antonio. (n.d.). Uso de temporizadores (timers). Recuperado de <https://picferalia.blogspot.com/2012/06/uso-de-temporizadores-timers.html>
- ▶ Perez, M. M. (n.d.). Timers AVR atmega8. Recuperado de <https://es.scribd.com/presentation/174029336/Timers-Avr-Atmega8>
- ▶ Matriz 8x8 - Mensaje con Desplazamiento (Scroll). (n.d.). Recuperado de <https://electgpl.blogspot.com/2013/11/matriz-8x8-mensaje-con-desplazamiento.html>
- ▶ Modos de direccionamiento de programa y datos. (n.d.). Recuperado de http://www.sc.ehu.es/sbweb/webcentro/automatica/web_avr/archivos/Manual_AT90S8515/Arquitectura/modos_direccionamiento.htm